VenturePulse: Predicting Startup Success from Structured Signals, Text Embeddings and Comparable Startups

Orchu Shirisha (23K91A05H6)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Mudavath Raghavendar (23K91A05G1)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Majgi Vaishnavi (23K91A05E0)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Polishetty Poojitha (23K91A05J4)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

P Prasanna (guide)
[DESIGNATION], Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

***Abstract*---**Most startups fail, yet investors and founders still judge early prospects largely by intuition. This paper builds and measures VenturePulse, a web application that estimates whether an early-stage startup will end in SUCCESS (acquisition or IPO) or FAILURE (closure). It combines 18 structured company fields with an optional business description embedded by a sentence encoder, predicts with a class-weighted classifier, and shows the comparable historical startups and the feature drivers behind each estimate. Four classifiers are compared with five repeated stratified 80/20 splits. On the public 922-row start-up table, Random Forest and XGBoost lead with balanced accuracy 0.746 and ROC-AUC 0.816 and 0.806, ahead of Logistic Regression (0.709), but accuracy stays near 0.78 and does not reach the 85-92 % reported in the base paper. Removing four fields that are recorded around the outcome lowers ROC-AUC by about 0.1. The shipped model was trained on a synthetic fallback table, and the text channel could not be tested on real data; both limits are stated.

***Index Terms*---**business analytics, class imbalance, feature importance, machine learning, random forest, similar-case retrieval, startup success prediction.

I. INTRODUCTION

Startups drive innovation and growth, but most fail within a few years because of weak funding, poor market fit, inexperienced management and competition [1], [2]. Investors and founders mostly decide on experience and intuition, which is subjective and inconsistent [1]. Structured data on funding, team, sector and location exist from the earliest stage, and machine-learning classifiers can learn which combinations of them preceded success [3], [4]. Ensembles such as Random Forest [5] and gradient boosting [6] handle the mixed categorical and numeric nature of such data well, but published studies leave four problems open: results are reported without dataset sizes or per-model tables [1], information recorded after the outcome can leak into the features [7], [8], the successful class is imbalanced [9], and a bare probability gives a founder nothing to act on.

This work therefore builds a complete, locally running system and measures it carefully. The contributions are: (1) VenturePulse, a FastAPI and React application that returns an outcome estimate, the comparable historical startups and the feature drivers for every request; (2) a fused representation of scaled structured features and MiniLM text embeddings with balanced, training-only cosine retrieval, adapted from the retrieval idea of [2], [10] without a language model; (3) a reproducible benchmark on the real 922-row public table with ten studies covering models, imbalance, thresholds, calibration, leakage-like fields and retrieval-only prediction; and (4) findings reported as found, including where the base paper's accuracy level is not reproduced. Section II reviews related work, Section III the method, Section IV the results, and Section V concludes.

A. System Hardware

All training, evaluation and latency measurements ran on one desktop computer: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, CPU only (no GPU).

B. Software Requirements

Python 3.11.9, scikit-learn 1.5.2, pandas 2.2.3, XGBoost 2.1.3, sentence-transformers 3.3.1, torch 2.5.1 (CPU), FastAPI 0.115.6, uvicorn 0.34.0; frontend React 18, Vite 6, Tailwind CSS 4, Recharts. Models use scikit-learn [11] and XGBoost [6]; the sentence encoder is all-MiniLM-L6-v2 [12].

II. LITERATURE SURVEY

Base paper. Panjwani et al. [1] build a Flask-based startup success predictor. Funding history, team size, industry and location features come from Crunchbase and Kaggle data; missing values are imputed by mean or mode, categories are label or one-hot encoded, numbers are standardised, and ADASYN oversampling [13] balances the classes. Logistic Regression, Random Forest, SVM and Gradient Boosting are trained on a 70:30 split and scored by accuracy, precision, recall, F1 and ROC-AUC. The paper reports that Random Forest and Gradient Boosting are best, with accuracy between 85 and 92 %, that SVM is accurate but slower, and that funding amount, rounds, team experience, industry and location matter most. It gives neither the dataset size nor a table of per-model scores. Its own limitations are scarce real-time data, imbalance and external market conditions, and it names look-ahead bias as a risk [7]. This project keeps the model set, the metrics and the

web tool, and adds retrieval of comparable startups, explanations, leakage checks and a protocol that reports spread over repeated splits.

Machine-learning prediction. Earlier studies compare Random Forest, Bayesian networks, SVM and boosting on funding and milestone features [3], [4]; as summarised by [1], ensembles exceed 80 % accuracy [4] and an ensemble over several sources reaches up to 90 % [14]. PCA-enhanced models improve classification [15], and a bias-free design avoids look-ahead bias [7]. **Retrieval and text.** Maarouf et al. [2] retrieve the k most similar past startups and show them with their outcomes to a language model; on 4,034 Crunchbase startups (35.15 % successful) kNN in-context learning reaches balanced accuracy 0.713 at k=50, against 0.631 for logistic regression, 0.634 for Random Forest and 0.618 for XGBoost, while a fused language model trained on 20,172 startups [16] reaches 0.743. Their study uses five repeated stratified 80/20 splits, masks company names to prevent memorisation, finds that performance saturates near 50 examples, and shows that structured fields alone are slightly better than text at very small k. **Interpretable and temporal models.** Mashhadi et al. [17] fit preprocessing on a 2010-2019 development window and apply it unchanged to later cohorts, weight classes by inverse prevalence, and compare logistic regression with Random Forest, XGBoost, LightGBM and CatBoost by PR-AUC with AUROC as tie-breaker. They obtain AUROC 0.817 for next funding (LightGBM), 0.872 for exit (Random Forest) and 0.921 for patenting (Random Forest); SHAP names time since the last round and age as first-order drivers, and reliability curves show optimism at high scores. **Economic evidence.** Judge disagreement in 67 venture competitions predicts success [18], and a survey of 120 respondents links venture capital with startup success [19].

TABLE I
RELATED WORK: APPROACH, DATA, REPORTED RESULT, LIMITATION

Work	Approach	Data	Reported result	Limitation
Panjwani [1] (base)	LR, RF, SVM, GB; ADASYN; 70:30	Crunchbase / Kaggle, size not stated	Accuracy 85-92 % (best)	No dataset size or table
Maarouf [2]	kNN in-context learning with a language model	4,034 startups, 35 % success	Bal. acc. 0.713 (k=50)	Needs an LLM and text
Mashhadi [17]	LR, RF, XGBoost, LightGBM; temporal splits	Crunchbase panel 2010-2023	AUROC 0.817-0.921	Panel not available here
Bidgoli [4]	RF, GB, SVM, LR compared	as summarised by [1]	Ensembles above 80 %	Not re-read
Zbikowski [7]	Bias-free prediction	Crunchbase	Look-ahead bias shown	Not re-read
Ross [14]	Ensemble of several sources	as summarised by [1]	Up to 90 %	Not re-read
Gius [18]	Judge disagreement	67 venture competitions	Predicts success	Not a classifier
Bhandari [19]	Survey regression	120 respondents	VC linked to success	Association only

Research gap. The surveyed work applies ensembles to startup success but rarely reports dataset sizes, controls fields recorded after the outcome, treats the imbalance explicitly, or tells the user which drivers and which comparable startups support a prediction. This work closes that gap with an application and with measurements of each of these points.

III. METHODOLOGY

Fig. 1 shows the three tiers. The browser interface (React 18 with Vite) calls a FastAPI service, which loads the saved artefacts once and calls the preprocessing, embedding, retrieval and prediction modules. State is held in files and in memory; no SQL database is used.

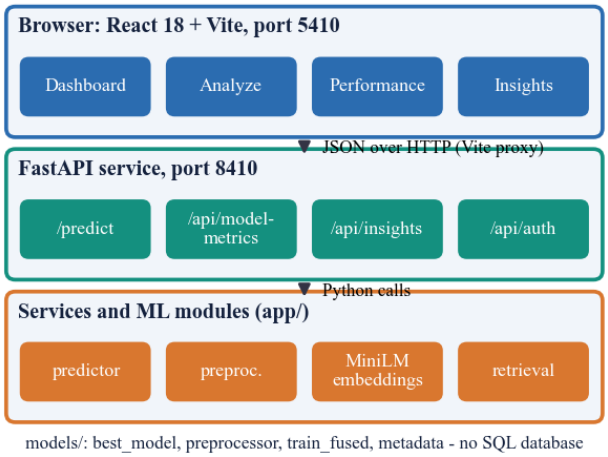


Fig. 1. System architecture: browser, FastAPI service, services and ML modules, and the saved model artefacts.

Data. Two tables are used and never mixed. The shipped application serves a 1,400-row synthetic fallback table (595 SUCCESS, 805 FAILURE) with the schema of the public table and template-generated descriptions, because the public download needs a login when the models were built; the wording of its success and failure templates differs, so its text channel leaks the label by construction. All real measurements use the public Kaggle 'Startup Success Prediction' table [20]: 922 rows after de-duplication (596 acquired = SUCCESS, 326 closed = FAILURE, 64.6 % success), US companies, structured columns only and no text. Nine numeric fields (funding total and rounds, ages at first and last funding and milestone, relationships, milestones, average participants), seven flags (VC, angel, rounds A to D, top-500) and the categorical state and industry give 37 encoded features plus log funding.

Feature construction. The public table's category code is mapped to the application's ten industry names (for example software, web and enterprise to Software; biotech to Biotech; medical and health to HealthTech) and the rest to Other, and the ten most frequent states are kept with the others grouped as Other. Funding velocity is approximated by the ages at first and last funding and the number of rounds; a log transform of total funding reduces the effect of a few very large rounds. No team-experience or market-trend field exists in either table, so the objective of engineering such features could only be met in part.

Preprocessing. Numbers are median-imputed and standardised, flags are most-frequent imputed, categories are

one-hot encoded with unknown values ignored, and the target is mapped (acquired or IPO to 1, closed to 0). Preprocessing is fitted on the training part only. Descriptions are cleaned and the company name is masked, to prevent memorisation [2]. Each description (at most 2,000 characters) is embedded by all-MiniLM-L6-v2 into a 384-d vector t [12]; if no description is given an industry sentence is embedded instead.

Models. Logistic Regression (L-BFGS, balanced weights, C=1), a Decision Tree (depth 8, balanced), a Random Forest [5] (300 trees, minimum leaf 2, balanced) and XGBoost [6] (400 trees, depth 6, learning rate 0.05, subsample 0.8) are trained; HistGradientBoosting is added in the real-data study. The application serves the model with the best balanced accuracy, because the classes are unequal and plain accuracy rewards the majority class. With s the scaled structured vector and t the text vector, the fused representation and the success probability are

$$x_f = [\alpha\, s,\ (1-\alpha)\, t] \,/\, \|[\alpha\, s,\ (1-\alpha)\, t]\|, \quad \alpha = 0.5, \qquad (1)$$

$$P(\text{success} \mid x) = 1 \,/\, (1 + \exp(-(w \cdot x + b))), \qquad (2)$$

shown for the linear model. Evidence is retrieved by cosine similarity against training rows only, with a balanced quota of k/2 SUCCESS and k/2 FAILURE neighbours, k in {10, 20, 30, 50} [10]; the similarity-weighted vote over the k retrieved neighbours is

$$p(x) = \Sigma_i\, sim(x, x_i)\, y_i \,/\, \Sigma_i\, sim(x, x_i), \quad i \text{ over the } k \text{ neighbours.} \qquad (3)$$

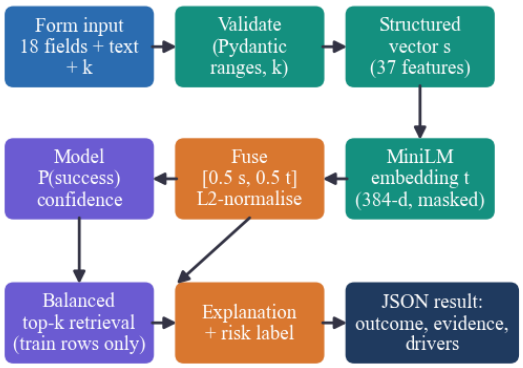


Fig. 2. Main flow of one prediction request (POST /predict).

The prediction procedure is: (1) validate the payload (funding between 0 and 10^10 USD, rounds up to 20, relationships up to 200, milestones up to 50, description at most 2,000 characters, k in the allowed set); (2) build one input row and derive log funding; (3) transform it with the fitted preprocessor; (4) embed the description; (5) fuse and normalise; (6) obtain P(success) and the larger class probability as confidence; (7) retrieve k balanced neighbours from the stored training vectors; (8) assemble the explanation and the risk label; (9) return the JSON result. Invalid input gets HTTP 422, an internal failure HTTP 500 with a message.

Application. The React interface has a landing page, a demo sign-in, a dashboard of the training data, the Analyze form and result screen, a Model performance page and an Insights page, all served from the same FastAPI back end through a proxy; the original server-rendered pages remain available. Sign-in uses an HMAC-signed, expiring token and only protects the demonstration screens; it is not a multi-user account system. If the saved artefacts are missing the service trains them on first start, so a clean checkout runs without manual steps.

Explanation and risk. The five largest signed coefficient or importance terms are returned with the retrieval statistics (neighbours per class, top similarity, mean similarity to each class). The label is 'Lower risk' for a SUCCESS verdict with confidence of at least 0.70, 'Moderate risk' from 0.55, otherwise 'Higher uncertainty'. The confidence is the larger class probability and is not a guarantee.

$$BA = (TPR + TNR) / 2, \quad F1 = 2\,P\,R \,/\, (P + R). \qquad (4)$$

Design decisions. Logistic Regression is served on the synthetic table because it has the best balanced accuracy there, and its signed weights give a direct explanation; the ensembles are compared but not served, because the real-data study was run after the application was built. Retrieval searches training rows only and uses a balanced quota so that a majority class cannot crowd out the minority neighbours, and nothing leaves the computer: no external prediction service or key is used. Artefacts are plain files loaded once into memory, which keeps start-up simple at the cost of single-user operation.

Evaluation protocol. Five repeated stratified 80/20 splits (seed 42) give the mean and standard deviation of accuracy, balanced accuracy, precision, recall and F1, plus ROC-AUC [9] for the real table; a 70:30 protocol with five seeds mirrors the base paper. The same code and hyper-parameters run on the real table through a separate script, which writes its results to a JSON file read by this paper. Implementation and testing: the back end is about 1,350 lines of Python, the front end about 3,000 lines of JavaScript, and 51 automated checks (7 unit tests, 15 smoke checks and 29 API edge cases) pass.

IV. RESULTS

Test conditions. The table is the public 922-row table with 596 successful and 326 failed companies; each of the five stratified 80/20 splits tests on 185 companies. Models are class-weighted and use the 0.5 threshold; values are mean ± standard deviation over the splits. Table II lists the results; the first block uses all fields, the second drops four fields recorded around the outcome (relationships, milestones, and the ages at first and last milestone), the third is a classifier-free retrieval vote, and the last row is the shipped model on its synthetic table.

TABLE II
RESULTS ON THE REAL 922-ROW TABLE (5 X 80/20 SPLITS)

Model	Acc.	Bal. acc.	F1	AUC
Logistic Regression	0.698	0.709±0.025	0.743	0.789
Decision Tree	0.717	0.703±0.031	0.774	0.742

Random Forest	0.773	0.746±0.031	0.826	0.816
XGBoost	0.775	0.746±0.026	0.829	0.806
HistGradientBoosting	0.761	0.727±0.023	0.820	0.806
RF, no post-hoc fields	0.718	0.677±0.017	0.789	0.719
XGBoost, no post-hoc fields	0.694	0.644±0.033	0.775	0.699
Retrieval vote only (k=10)	0.750	0.714±0.014	0.813	0.769
Shipped LR (synthetic, fused)	0.751	0.749±0.025	0.716	n/r

Random Forest and XGBoost are best, both with balanced accuracy 0.746, accuracy 0.773 and 0.775 and F1 near 0.83; Random Forest has the highest ROC-AUC (0.816). They are within one standard deviation of HistGradientBoosting (0.727) and clearly above Logistic Regression (0.709, ROC-AUC 0.789) and the Decision Tree (ROC-AUC 0.742). With the base paper's 70:30 split over five seeds, Random Forest scores accuracy 0.796 and Logistic Regression 0.729. The F1 values under that protocol are 0.851 (Random Forest), 0.842 (HistGradientBoosting) and 0.781 (Logistic Regression), with ROC-AUC 0.828, 0.824 and 0.795. The ranking of ensembles over the linear baseline therefore agrees with [1], but its 85-92 % accuracy is not reached, and since that paper states neither its data nor a table, the gap cannot be explained.

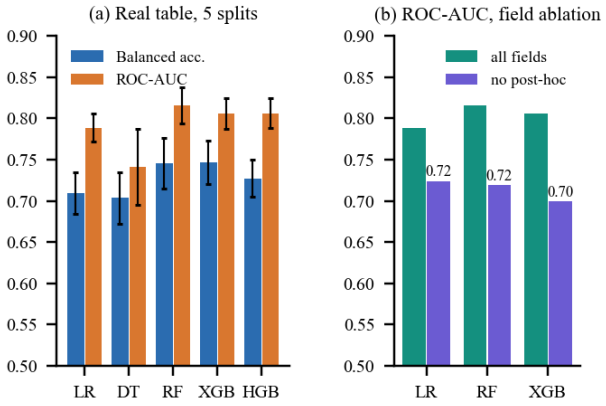


Fig. 3. (a) Balanced accuracy and ROC-AUC of five models on the real table; (b) ROC-AUC with all fields and without the four post-hoc fields.

Leakage-like fields. relationships, milestones and the two milestone ages can be recorded after the outcome window, and the table has no dates to remove them [7], [8]. Dropping them lowers ROC-AUC from 0.816 to 0.719 for Random Forest and from 0.806 to 0.699 for XGBoost, and removes the ensembles' lead over Logistic Regression (0.724). The headline numbers are therefore an upper bound for genuine early-stage prediction. **Features.** relationships ranks first by Random Forest Gini (0.162) and by permutation importance (drop of 0.060 in ROC-AUC), followed by milestones, the age of the last milestone, funding amount and age at first funding; the Logistic Regression weights are led by state, top-500 and industry indicators, so the two rankings differ.

TABLE III
IMBALANCE HANDLING ON THE REAL TABLE (SUCCESS IS THE MAJORITY CLASS)

Model, strategy	Bal. acc.	Recall	F1	AUC
LR, none	0.714±0.020	0.815	0.805	0.791
LR, class weights	0.709±0.025	0.673	0.743	0.789
LR, oversampling	0.712±0.022	0.683	0.749	0.783
RF, none	0.750±0.025	0.878	0.843	0.820
RF, class weights	0.746±0.031	0.837	0.826	0.816
RF, oversampling	0.749±0.026	0.858	0.836	0.816

Imbalance and threshold. SUCCESS is the majority class here (64.6 %), the reverse of the rare-success assumption [9]. As Table III shows, class weights and oversampling did not raise ROC-AUC (Random Forest 0.820 without any treatment, 0.816 with either); they trade recall of the success class for precision (Logistic Regression recall 0.815 unweighted, 0.673 weighted). For weighted Logistic Regression, balanced accuracy is flat (0.709-0.714) for thresholds between 0.30 and 0.50 while recall rises from 0.673 to 0.875, and calibration is poor: in the 0.3-0.4 bin the predicted success probability is 0.348 but 0.648 of companies succeeded. **Retrieval.** The retrieval vote alone, with no trained model, scores balanced accuracy 0.70-0.71 and ROC-AUC 0.77-0.79 for every k, equal to Logistic Regression, so the retrieved evidence is as informative as the linear model; k matters little on real data, whereas on the synthetic table accuracy grows from 0.654 (k=10) to 0.698 (k=50).

Synthetic data. The shipped Logistic Regression reaches balanced accuracy 0.749 on its synthetic table (structured only 0.714, text only 0.599, combined 0.749). Because the synthetic text leaks the label, this does not show that text helps; the text half of the fusion is untested on real data. Trained on the synthetic rows and tested on all 922 real rows, Logistic Regression gives balanced accuracy 0.673 and ROC-AUC 0.745.

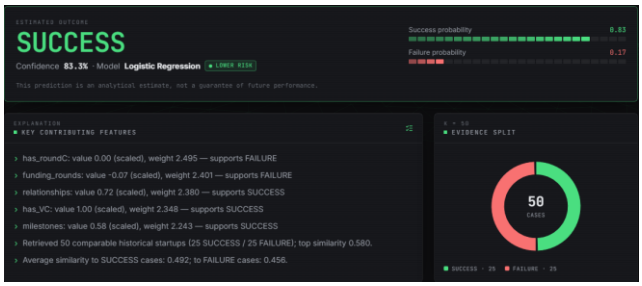


Fig. 4. Output screen (Analyze): SUCCESS, confidence 83.3 %, Lower risk, key drivers and the 25 + 25 comparable startups, for a submitted software startup.

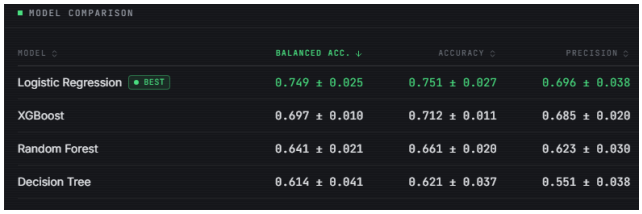


Fig. 5. Output screen (Model performance): model comparison and charts of the shipped model on its synthetic table.

Fig. 4 is a capture of the Analyze result for a software startup in California with USD 2.5 million raised over two rounds, six relationships, three milestones, venture backing and a round A: the model returns SUCCESS with 83.3 % confidence and 'Lower risk', cites has_roundC, funding rounds, relationships, venture backing and milestones as its largest terms, and lists 50 comparable startups (25 and 25) with top similarity 0.580. This screen is served by the model

trained on the synthetic table, so its confidence reflects that table and not the real one. Fig. 5 shows the Model performance page of the same model. Fig. 6 plots the permutation drivers on the real table and the threshold sweep of Logistic Regression.

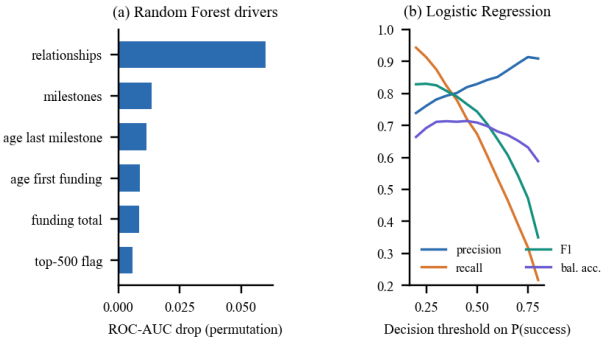


Fig. 6. (a) Top permutation drivers of Random Forest on the real table; (b) precision, recall, F1 and balanced accuracy of Logistic Regression against the decision threshold.

Size and industries. Random Forest with 50, 100, 300 and 600 trees reaches balanced accuracy 0.742, 0.744, 0.746 and 0.741, so the 300 trees used are adequate and more trees add nothing. Accuracy by industry on one test split is 0.73 for Software (106 companies) and 0.89 for Media (35), but groups of six or ten companies cannot be read.

Comparison and cost. Maarouf et al. [2] score 0.713 balanced accuracy for retrieval with a language model and 0.618-0.634 for supervised baselines on 4,034 startups with 35 % successes; this work measures 0.709 (Logistic Regression) to 0.746 (ensembles) on a different table with no text and 65 % successes, so the numbers are not comparable. The temporal study [17] reports AUROC 0.817-0.921 with leakage-safe splits, whereas this work has no temporal split. The server answers POST /predict in a median 148 ms (k=10) and 198 ms (k=50), 95th percentile about 590 ms, and read endpoints in 3-15 ms (CPU, local requests).

Testing and objectives. The 7 unit tests cover health, dataset loading, preprocessing with missing values, embedding and fusion, balanced retrieval, prediction and pages; the 15 smoke checks run against the live server; and the 29 edge cases confirm that a wrong k, negative or oversized funding, too many rounds, an over-long description, a non-numeric value and a tampered token are rejected with 422 or 401, while an empty body, an unknown industry or state, and maximum values are accepted. Against the six objectives, collecting data and engineering features are only partly met (the real table lacks team-experience and market fields, and funding velocity is approximated by the ages at first and last funding), while model comparison, imbalance handling, driver analysis and the full metric set are met.

Calibration and confidence. The class-weighted Logistic Regression under-states the success probability on this majority-success table: observed success rates exceed the mean predicted probability in every bin below 0.9 (for example 0.436 against 0.252 in the 0.2-0.3 bin and 0.755 against 0.550 in the 0.5-0.6 bin), and only the top bin (0.953 predicted, 0.931 observed) is close. The 'confidence' shown by the application is the larger class probability and the three risk labels are fixed rules, so neither is a calibrated statement; isotonic recalibration, which [17] recommends when calibrated probabilities are needed, was not applied.

Reproducibility. The real-table study of five models, six imbalance settings, two protocols, the threshold sweep and the transfer test runs in 139 s on the CPU, and the leakage, retrieval and industry studies in 14 s; every number here is read from the JSON files they write. Hyper-parameters were fixed from the application and not tuned per model, so the ranking of the three boosted or bagged ensembles, which differ by less than one standard deviation, should not be over-read.

Failure cases and limits. Random Forest misses 19.6 of 120 successes and wrongly accepts 22.4 of 65 failures per test fold; small industries are unreliable (Biotech and CleanTech groups of six companies, accuracy 0.33 and 0.50). Test folds have only 185 rows, the data are US-only and have no founder or market fields, and no significance test, load test or user study was run.

V. CONCLUSION

VenturePulse meets its engineering goals: a local application that returns an outcome estimate with comparable startups and feature drivers, backed by 51 passing checks. Against the objectives, four classifiers were compared, imbalance was handled and examined, the features separating the classes were identified, and accuracy, precision, recall, F1 and ROC-AUC were measured. The key finding is that ensembles beat Logistic Regression on the real table (balanced accuracy 0.746 against 0.709) but fall short of the 85-92 % accuracy of the base paper, and that about a tenth of ROC-AUC rests on fields recorded around the outcome. Limitations are the synthetic fallback in the shipped model, the untested text channel, the absence of founder, team-experience and market data, and the absence of a temporal split. Future work: temporal validation on dated data, founder and market features, SHAP-based explanations, calibrated probabilities, and testing text retrieval on real descriptions. A larger dated dataset, such as a firm-quarter panel [17], would allow leakage-safe evaluation, and a user study with founders and investors would show whether the comparable startups and drivers make an estimate easier to act on, which this work did not test.

REFERENCES

[1] M. Panjwani, P. Shrivastava, P. Hande, G. Meshram, N. Verma and D. Gotmare, "Startup success prediction using machine learning and ensemble techniques," Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET), vol. 14, no. IV, Apr. 2026, doi: 10.22214/ijraset.2026.80226.

[2] A. Maarouf, A. Bakiaj and S. Feuerriegel, "Predicting startup success using large language models: A novel in-context learning approach," arXiv:2601.16568, Jan. 2026.

[3] A. Krishna, A. Agrawal and A. Choudhary, "Predicting the outcome of startups: Less failure, more success," Northwestern University, 2016.

[4] M. Razaghzadeh Bidgoli, I. Raeesi Vanani and M. Goodarzi, "Predicting the success of startups using a machine learning approach," J. Innov. Entrepreneurship, vol. 13, 2024.

[5] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, Oct. 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, San Francisco, CA, USA, 2016, pp. 785-794.

[7] K. Zbikowski and P. Antosiuk, "A machine learning, bias-free approach for predicting business success using Crunchbase data," Inf. Process. Manage., vol. 58, no. 2, 2021.

[8] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Trans. Knowl. Discovery Data, vol. 6, no. 4, Art. no. 15, 2012.

[9] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263-1284, Sep. 2009.

[10] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Trans. Inf. Theory, vol. 13, no. 1, pp. 21-27, Jan. 1967.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP-IJCNLP), Hong Kong, 2019, pp. 3982-3992.

[13] H. He, Y. Bai, E. A. Garcia and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN), Hong Kong, 2008, pp. 1322-1328.

[14] G. Ross, S. Das, D. Sciro and H. Raza, "CapitalVX: A machine learning model for startup selection and exit prediction," J. Finance Data Sci., vol. 7, pp. 94-114, 2021.

[15] Y. Choi, "Startup success prediction with PCA-enhanced machine learning models," J. Technol. Manage. Innov., vol. 19, no. 4, 2024.

[16] A. Maarouf, S. Feuerriegel and N. Prollochs, "A fused large language model for predicting startup success," Eur. J. Oper. Res., 2025.

[17] S. Mashhadi, A. Saghezchi and V. G. Kashani, "Interpretable machine learning for predicting startup funding, patenting, and exits," arXiv:2510.09465, 2026.

[18] L. Gius, "Disagreement predicts startup success: Evidence from venture competitions," Strategy Sci., vol. 10, no. 2, pp. 93-108, Jun. 2025, doi: 10.1287/stsc.2024.0169.

[19] I. B. Bhandari, "Impact of venture capital on startup success and economic growth in Nepal," Nepalese J. Finance, vol. 12, no. 1, Jan. 2025.

[20] M. KC, "Startup success prediction," Kaggle dataset (public domain), https://www.kaggle.com/datasets/manishkc06/startup-success-prediction.